

satya@Elegance: ~/Desktop/OSFY:Articles

```
[satya]
./a.out
Enter the size: 3
Size of array : 12
[satya]
```

Figure 5: Output of the `sizeof_run.c`

For an explanation, let us consider the code snippet given below:

```
1 #include <stdio.h>
2 #include <stddef.h>
3
4 int main()
5 {
6     int array[] = {10, 20, 30, 40, 50};
7
8     size_t i;
9
10    for(i = 0; i < sizeof(array) / sizeof(array[0]);
11    i++)
12    {
13        //some code
14    }
```

In the example shown above, at line number 10, by using the `sizeof()` operator the number of elements is automatically computed.

The sizes of primitive data types in C are implementation defined. For example, the `sizeof(long)` on 32-bit architecture may vary from that on 64-bit architecture. So, when we decide statically the `sizeof(long)` as 4 bytes wide on 32-bit architecture, and when the same code is ported to 64-bit architecture, the results may go wrong. So, in order to avoid the portability issue, it is a best practice to use `sizeof()` to compute the `sizeof` variables or arrays, depending on the exact size of a particular data type.

Case 2: To allocate a block of memory dynamically of a particular data type

In case of dynamic memory allocation of an array, `sizeof()` plays an important role. For example, if we want to allocate a block of memory that is big enough to hold '5' integers in an array, `sizeof` comes in handy and is a great help, since we do not know the exact `sizeof(int)` to dynamically allocate the memory using `malloc` function for a particular architecture.

```
int *iptr = malloc(5 * sizeof(int));
```

In the above example, we are *allocating* the block of memory, which is equal to the number of bytes of type

```
//Declaring the variable length array
int array[size];
00484a3: 8b 4d d8      mov     -0x20(%ebp), %ecx
00484a5: 8d 41 12      lea     -0x11(%ecx), %eax
00484a7: 89 45 dc      mov     %eax, -0x24(%ebp)
00484a9: 89 c8         mov     %ecx, %eax
00484ab: c1 e6 02      shl     $0x2, %eax
00484ad: 03 11 02      lea     0x3(%eax), %edx
00484af: b8 10 00 00  mov     $0x10, %eax
00484b1: 83 e8 01      sub     $0x1, %eax
00484b3: 01 d0         add     %edx, %eax
00484b5: ba 00 00 00  mov     $0x10, %edx
00484b7: ba 00 00 00  mov     $0x10, %edx
00484b9: 77 f6        div     %esi
00484bb: 6b c8 10      imul    $0x10, %eax, %eax
00484bd: 29 c4        sub     %eax, %esp
00484bf: 8d 44 24 08   lea     0x8(%esp), %eax
00484c1: 83 e0 01      and     $0x2, %eax
00484c3: c1 e6 02      shl     $0x2, %eax
00484c5: c1 e6 02      shl     $0x2, %eax
00484c7: c1 e6 02      shl     $0x2, %eax
00484c9: 89 45 e0      mov     %eax, -0x20(%ebp)

//Finding the size of array
array_size = sizeof(array);
00484dd: 89 c8         mov     %ecx, %eax
00484df: c1 e6 02      shl     $0x2, %eax
00484e1: 89 45 e4      mov     %eax, -0x1c(%ebp)

printf("Size of array : %d\n", array_size);
00484e7: 8b 45 e0      mov     -0x1c(%ebp), %eax
00484e9: 89 44 24 04   mov     %eax, 0x4(%esp)
00484eb: c7 04 24 b3 80 04 08  movl    $0x80485b3, (%esp)
00484ed: e8 36 fe ff ff  call    0048330 <printf@plt>

return 0;
00484f1: d8 00 00 00  mov     $0x0, %eax
00484f3: 89 dc        mov     %eax, %esp
```

Figure 6: Assembly code generated by the compiler, for variable length array

`int`, multiplied by 5, ensuring sufficient space for all five `ints` is allocated.

Case 3: To determine the `sizeof` compound data types

Sometimes, it is very difficult to predict the sizes of compound data types such as structures, due to structure padding, and to predict the size of unions. `sizeof()` is of great use here.

Cases when `sizeof()` will not work

The `sizeof` operator will not work when applied to the following cases:

- A bit field
- A function type
- An incomplete type (such as `void`)
- Zero-sized array (except in GCC, which supports zero-sized structures as a GNU extension).

`sizeof()` and incomplete data types

An incomplete type in C is one that describes an identifier, but lacks the information needed to determine the size of the identifier.

Examples of incompletely defined types are:

1. An array type whose dimensions have not yet been specified
2. A structure type whose members have not yet been specified

Illustration: An array whose dimensions are not specified

File1.c	File2.c
int array[10];	extern int array;